

Proyecto GestorFFEOE “Bienvenidos a segundo”

Este proyecto está planteado para desarrollarse por parejas o tríos.

El **objetivo** es hacer una aplicación web funcional para gestionar todo lo relacionado con la asignación de empresas a alumnado para el periodo de prácticas en el instituto.

Stack Tecnológico (A elección del grupo de alumnos)

- **Backend:** PHP (Vanilla o Symfony) **O** Python (Django/FastAPI).
 - **Frontend:** JavaScript (React, Vue o Vanilla JS con componentes).
 - **Base de Datos:** Relacional (MySQL/PostgreSQL)
 - **Git:** Uso obligatorio de un flujo de trabajo..
-

Requisitos Funcionales por Roles

1. Administrador / Coordinador

- **Gestión de Ciclos:** Crear ciclos (nombre, año inicio, año fin).
- **Gestión de Profesores:** Asignar profesores a ciclos.
- **Configuración Global:** Definir periodos de asignación.

2. Profesores (Gestores de grupo)

- **Importación Masiva:** Subir un **CSV** para matricular a todos los alumnos de una clase de golpe. Hacer lo mismo para subir empresas con un csv.
- **Seguimiento de Empresas:** Un profesor puede registrar que ha contactado un empresa un día y hora concreto.
- **Gestión de Plazas:** Indicar cuántos alumnos de qué ciclo acepta una empresa.
- **Asignación Dinámica:** Una vista tipo "drag & drop" o selector donde asocia un alumno a una plaza disponible de una empresa.

3. Alumno

- **Perfil y CV:** Subir su currículum (PDF) y actualizar sus datos de contacto.
- **Dashboard:** Ver el estado de su asignación (Pendiente / Asignado a [Nombre Empresa] / Tutor Laboral: [Nombre]).

4. Datos de Empresa (Entidades)

- **Empresa:** Dirección, Web, Persona contacto, Email, Teléfono.
- **Personal:** 1 Responsable legal (DNI) y N Tutores laborales (DNI, Teléfono).

Dinámica de Trabajo en Equipo (Parejas/Tríos)

Para que el proyecto cuente como un proyecto real, hay que colaborar:

- **Protección de Ramas:** La rama `main` debe estar protegida. Nadie puede subir código directamente.
- **Code Reviews (PR):** Para que una funcionalidad se sume al proyecto, otro miembro del equipo debe revisar el código y dejar al menos un comentario de mejora o aprobación.
- **Ejemplo de reparto de Tareas:**
 - *Alumno A:* Especialista en Backend (API, Base de datos, CSV).
 - *Alumno B:* Especialista en Frontend (Vistas, Formularios, Dashboard).
 - *Alumno C (si es trío):* Especialista en QA y Lógica de IA (Tests, integraciones, validaciones de seguridad).

Requisitos del Proyecto

1. Estructura y Código

- **Arquitectura:** Separación clara entre Frontend y Backend (API-First).
- **Estructura de Código:** Uso estricto de **clases y namespaces** en PHP y **entornos virtuales con módulos** en Python. Nada de código "espaguete" en un solo archivo.
- **Clean Code:** Nombres de variables semánticos, funciones de menos de 20 líneas y tipado en Python/PHP.
- **Documentación:** Uso de **Swagger/OpenAPI** para documentar los endpoints.
- **Requisito extra:** Implementar **JWT (JSON Web Tokens)** para la autenticación entre el front y los dos backends. Investiga sobre qué es esto!

Uso de IA

Sobre las Unit Test:

Puedes usar IA para generar las pruebas unitarias del backend, pero es tu responsabilidad asegurar que los tests cubren los casos de error (ej. intentar asignar un alumno a una empresa sin plazas libres).

Auditoría:

Usar la IA para analizar fallos de seguridad. Verificar que lo detecta correctamente.

Sobre el uso de agentes IA:

Está recomendado con mucho énfasis. Con tanto énfasis que se requiere analizar el código que genera para verificar que está haciendo lo que debe.

Si tu pregunta al leer esto es "es que yo no se php, python o javascript", pues felicidades, vas a aprender usándolo.